

06 · Aggregation and GROUP BY

Why this matters in interviews. "WHERE vs HAVING", "COUNT(*) vs COUNT(col)", "find duplicate rows", "department with the most employees" — aggregation questions appear in nearly every SQL round, and they punish anyone who mixes up the order things are evaluated in.

In this chapter: aggregate functions · GROUP BY · WHERE vs HAVING · counting patterns (including duplicates and empty groups) · conditional aggregation · ROLLUP / CUBE / GROUPING .

6.1 Aggregate functions

An **aggregate function** performs a calculation on a **set of values** and returns a **single value**. They are most often used with the GROUP BY clause, which splits the result set into groups and lets the aggregate return one value **per group**.

Function	Returns
MIN()	the smallest value within the selected column
MAX()	the largest value within the selected column
COUNT()	the number of rows in a set
SUM()	the total sum of a numerical column
AVG()	the average value of a numerical column
MEDIAN() (Oracle)	the middle value

```
SELECT COUNT(*)    AS headcount,
       MIN(salary)  AS lowest,
       MAX(salary)  AS highest,
       SUM(salary)  AS payroll,
       AVG(salary)  AS average,
       MEDIAN(salary) AS median
FROM emp;
```

Result

headcount	lowest	highest	payroll	average	median
10	20000	90000	500000	50000	47500

MEDIAN (the middle of the sorted salaries: 45,000 and 50,000 → 47,500) differs from the average (50,000) — a favourite follow-up question. MIN / MAX also work on **text and dates**: MIN(hire_date) is the earliest hire, MAX(emp_name) the alphabetically last name.

Aggregate functions ignore NULL values — except COUNT(*) . (NULLs in a column are skipped by SUM , AVG , MIN , MAX and COUNT(column) .)

The three flavours of COUNT

Expression	Counts
COUNT(*)	all rows , whatever their values
COUNT(column)	rows where column is not NULL
COUNT(DISTINCT column)	the number of different non-NULL values

```
SELECT COUNT(*)           AS all_rows,  
       COUNT(comm)        AS with_commission,  
       COUNT(dept_id)      AS with_dept,  
       COUNT(DISTINCT dept_id) AS distinct_depts,  
       COUNT(DISTINCT job)  AS distinct_jobs  
FROM emp;
```

Result

all_rows	with_commission	with_dept	distinct_depts	distinct_jobs
10	2	9	3	7

- 10 rows in total, but only **2** have a commission and **9** have a department (Jai has none).
- COUNT(DISTINCT dept_id) is 3 — departments 10, 20, 30; the NULL is not a value.

Trap. SUM of an empty set, or of only NULLs, is **NULL**, not 0. Write NVL(SUM(x), 0) if you need a number. COUNT of an empty set is 0.

Trap. AVG(comm) divides by the number of *non-NULL* rows (2), not by 10. To treat NULL as zero: AVG(NVL(comm, 0)). The two answers differ enormously — 4,000 versus 800.

```
SELECT AVG(comm) AS avg_ignoring_nulls, AVG(NVL(comm, 0)) AS avg_nulls_as_zero FROM emp;
```

Result

avg_ignoring_nulls	avg_nulls_as_zero
4000	800

6.2 GROUP BY

The GROUP BY statement **groups rows that have the same values into summary rows** — like "find the number of customers in each country". It is often used with the aggregate functions (COUNT() , MAX() , MIN() , SUM() , AVG()) to group the result-set **by one or more columns**.

```
SELECT    column_name(s)
FROM      table_name
WHERE     condition
GROUP BY  column_name(s)
ORDER BY  column_name(s);
```

The sample table

sales

product	category	price	quantity
Apple	Fruit	30	5
Banana	Fruit	10	10
Carrot	Veg	20	7
Apple	Fruit	30	8
Carrot	Veg	20	3

1 · Group by a single column

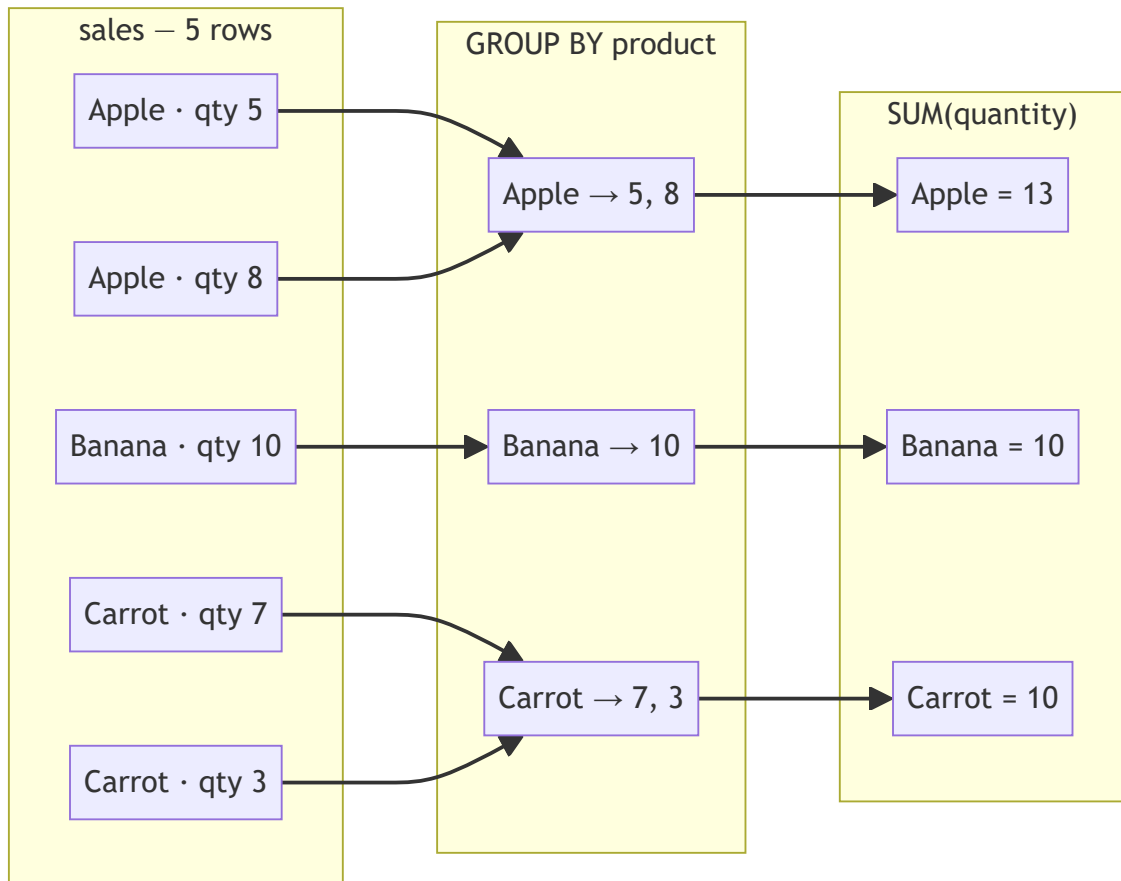
Total quantity sold per product.

```
SELECT product, SUM(quantity) AS total_quantity
FROM    sales
GROUP BY product;
```

Result

product	total_quantity
Apple	13
Banana	10
Carrot	10

What the database did — form **buckets** of rows with the same `product` , then collapse each bucket into one row:



2 · Group by multiple columns

Rows are grouped by the **combination** of values:

```
SELECT category, product, SUM(quantity) AS total_quantity
FROM sales
GROUP BY category, product;
```

Result

category	product	total_quantity
Fruit	Apple	13
Fruit	Banana	10
Veg	Carrot	10

3 · GROUP BY with ORDER BY

You can order the grouped results — even by an **aggregate** or its **alias** (ORDER BY runs after SELECT):

```
SELECT product, SUM(quantity) AS total_quantity
FROM sales
GROUP BY product
ORDER BY total_quantity DESC;
```

Result

product	total_quantity
Apple	13
Carrot	10
Banana	10

4 • GROUP BY with COUNT

```
SELECT category, COUNT(*) AS total_entries
FROM sales
GROUP BY category;
```

Result

category	total_entries
Fruit	3
Veg	2

5 • Group by an expression

You can group by an expression too — here, ignoring case:

```
SELECT UPPER(category) AS upper_cat, COUNT(*) AS n
FROM sales
GROUP BY UPPER(category);
```

Result

upper_cat	n
FRUIT	3
VEG	2

6 • GROUP BY with a JOIN

Count orders per customer:

```
SELECT c.name, COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;
```

Result

name	total_orders
Alice	2
Bob	1

(Charlie has no orders so an inner join drops him; §6.4 shows how to keep him with a `LEFT JOIN`.)

The two rules you must know

Rule 1. Every column in the `SELECT` list must either **appear in the `GROUP BY`** or be **inside an aggregate function**.

```
sql SELECT dept_id, emp_name, AVG(salary) FROM emp GROUP BY dept_id; -- ORA-00979: not a GROUP BY expression (which employee name should represent the group?)
```

Rule 2. `GROUP BY` comes **after `WHERE`** and **before `ORDER BY`**.

Things that surprise people

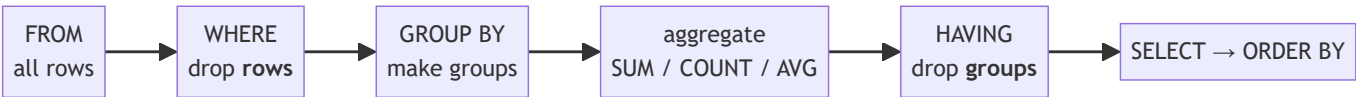
Situation	What happens
NULLs in the grouping column	all NULLs form one group (Jai's <code>dept_id</code> group)
Alias in <code>GROUP BY</code>	✗ not allowed in Oracle (up to 21c): <code>SELECT category AS cat, SUM(quantity) AS qty FROM sales GROUP BY category</code> — repeat the column, not the alias <code>cat</code> . (MySQL and PostgreSQL do allow it)
Ordinal in <code>GROUP BY</code> (<code>GROUP BY 1</code>)	✗ not allowed in Oracle (works in MySQL/PostgreSQL). Oracle <i>does</i> allow ordinals in <code>ORDER BY</code>
Grouping by a function	✓ <code>GROUP BY UPPER(category)</code> ; the same expression must appear in <code>SELECT</code>
A <code>GROUP BY</code> with no aggregate	acts like <code>DISTINCT</code>

6.3 WHERE VS HAVING

The `HAVING` clause was **added to SQL because the `WHERE` keyword cannot be used with aggregate functions**.

- `WHERE` filters **rows** — *before* grouping.
- `HAVING` filters **groups** — *after* aggregation.

```
SELECT  column_name(s)
FROM    table_name
WHERE   condition
GROUP BY column_name(s)
HAVING  condition
ORDER BY column_name(s);
```



`HAVING` — keep groups whose total exceeds 10:

```
SELECT product, SUM(quantity) AS total_quantity
FROM sales
GROUP BY product
HAVING SUM(quantity) > 10;
```

Result

product	total_quantity
Apple	13

Only *Apple* (13) survives: *Banana* and *Carrot* each total exactly 10.

WHERE — filter first, then group:

```
SELECT category, AVG(price) AS avg_price
FROM sales
WHERE price > 10
GROUP BY category;
```

Result

category	avg_price
Fruit	30
Veg	20

The *Banana* row (price 10) was dropped **before** averaging, so *Fruit*'s average is over *Apple*'s 30 only.

Both in one query — the textbook example: *list the number of employees in each department; only include departments with more than 1 employee; sorted high to low.*

```
SELECT dept_id, COUNT(emp_id) AS employees
FROM emp
GROUP BY dept_id
HAVING COUNT(emp_id) > 1
ORDER BY COUNT(emp_id) DESC;
```

Result

dept_id	employees
20	5
30	3

Notice that the NULL group (*Jai* alone) has 1 employee so **HAVING** removes it.

Mixing them correctly — average salary per department, counting only employees earning at least 30,000, and keeping only departments whose average is above 50,000:

```

SELECT dept_id, ROUND(AVG(salary)) AS avg_salary, COUNT(*) AS n
FROM emp
WHERE salary >= 30000          -- row filter, before grouping
GROUP BY dept_id
HAVING AVG(salary) > 50000     -- group filter, after aggregating
ORDER BY dept_id;

```

Result

dept_id	avg_salary	n
10	90000	1
20	52500	4

	WHERE	HAVING
Filters	individual rows	whole groups
Runs	before GROUP BY	after aggregation
Can use aggregates?	✗ no (ORA-00934: group function is not allowed here)	✓ yes
Can use plain columns?	✓	only grouped columns
Speed	faster — shrinks the data early	works on the (smaller) grouped set

Performance rule. If a condition doesn't need an aggregate, put it in `WHERE` , not `HAVING` — filtering rows early means less to group.

6.4 Classic patterns

Find the duplicates

Group by the column(s) that should be unique and keep groups of more than one:

```

SELECT salary, COUNT(*) AS how_many
FROM emp
GROUP BY salary
HAVING COUNT(*) > 1
ORDER BY salary;

```

Result

salary	how_many
35000	2
50000	2

The department with the most employees

Group, order by the count, take the first row:

```
SELECT d.dept_name, COUNT(e.emp_id) AS number_of_employees
FROM   emp e
JOIN   dept d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY number_of_employees DESC
FETCH FIRST 1 ROWS ONLY;
```

Result

dept_name	number_of_employees
Engineering	5

(If two departments tie, use `FETCH FIRST 1 ROWS WITH TIES`, or a `HAVING COUNT(*) = (SELECT MAX(...))` form — Chapter 08.)

Include groups that have zero rows

An inner join loses departments with no employees. Use a `LEFT JOIN` and count a column from the *many* side, not `COUNT(*)`:

```
SELECT d.dept_name, COUNT(e.emp_id) AS employees
FROM   dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY employees DESC, d.dept_name;
```

Result

dept_name	employees
Engineering	5
Sales	3
Management	1
HR	0

HR shows **0**. With `COUNT(*)` it would wrongly show **1** (the single NULL-extended row produced by the outer join). `COUNT(e.emp_id)` counts only real employees.

Conditional aggregation — a pivot with `CASE`

Put a `CASE` **inside** the aggregate to count or sum only the rows that match a condition — several conditions in one pass, side by side:

```

SELECT d.dept_name,
       SUM(CASE WHEN e.job = 'Developer' THEN 1 ELSE 0 END) AS developers,
       SUM(CASE WHEN e.job LIKE '%Manager%' THEN 1 ELSE 0 END) AS managers,
       SUM(CASE WHEN e.comm IS NOT NULL THEN e.comm ELSE 0 END) AS total_commission,
       SUM(CASE WHEN e.job = 'Developer' THEN e.salary ELSE 0 END) AS developer_pay
FROM   dept d
LEFT   JOIN emp e ON e.dept_id = d.dept_id
GROUP  BY d.dept_name
ORDER  BY d.dept_name;

```

Result

dept_name	developers	managers	total_commission	developer_pay
Engineering	2	1	0	100000
HR	0	0	0	0
Management	0	0	0	0
Sales	0	1	8000	0

Count *males and females per department* or *total pay of programmers per department* the same way (`SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END)`).

6.5 ROLLUP, CUBE and GROUPING — subtotals and grand totals

These are **advanced GROUP BY extensions** (Oracle, SQL Server, PostgreSQL) that add summary rows automatically.

- **ROLLUP** adds **subtotals and a grand total**, moving from the last listed column up to the first.
- **CUBE** adds **subtotals for all combinations** of the listed columns.

```

SELECT category, product, SUM(quantity) FROM sales GROUP BY ROLLUP(category, product);
SELECT category, product, SUM(quantity) FROM sales GROUP BY CUBE(category, product);

```

ROLLUP(category, product) on the `sales` table — a `NULL` in a column means "all values of this column":

```

SELECT category, product, SUM(quantity) AS total
FROM   sales
GROUP  BY ROLLUP(category, product)
ORDER  BY category, product;

```

Result

category	product	total
Fruit	Apple	13
Fruit	Banana	10
Fruit	NULL	23
Veg	Carrot	10

category	product	total
Veg	NULL	10
NULL	NULL	33

Reading it: each *category/product* line, then a **category subtotal** (product = NULL), and a final **grand total** (both NULL). `ROLLUP(a, b)` produces the groupings *(a, b)*, *(a)* and *()*.

`CUBE(category, product)` adds the "by product across all categories" totals as well — groupings *(a, b)*, *(a)*, *(b)*, *()*:

```
SELECT category, product, SUM(quantity) AS total
FROM sales
GROUP BY CUBE(category, product)
ORDER BY category, product;
```

Result

category	product	total
Fruit	Apple	13
Fruit	Banana	10
Fruit	NULL	23
Veg	Carrot	10
Veg	NULL	10
NULL	Apple	13
NULL	Banana	10
NULL	Carrot	10
NULL	NULL	33

`GROUPING(col)` tells a *subtotal* NULL (1) from a *real* NULL in the data (0) — essential to label the rows properly:

```
SELECT DECODE(GROUPING(category), 1, 'ALL CATEGORIES', category) AS category,
       DECODE(GROUPING(product), 1, 'ALL PRODUCTS', product) AS product,
       SUM(quantity) AS total
FROM sales
GROUP BY ROLLUP(category, product);
```

`GROUPING SETS ((category), (product))` computes exactly the groupings you list and nothing else.

Interview corner

Q. WHERE vs HAVING ? `WHERE` filters rows before grouping and cannot use aggregates; `HAVING` filters groups after aggregation. Use `WHERE` for anything that doesn't need an aggregate.

Q. COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column) ? All rows · non-NULL values · distinct non-NULL values.

Q. Why does `SELECT dept_id, emp_name, AVG(salary) ... GROUP BY dept_id` fail? `emp_name` is neither grouped nor aggregated — `ORA-00979`. Add it to the `GROUP BY` or wrap it in an aggregate.

Q. Do aggregate functions count NULLs? No, except `COUNT(*)`.

Q. How do you find duplicate rows? `SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1;`

Q. How do you list a department that has no employees with a count of zero? `LEFT JOIN` from the department table and `COUNT(e.emp_id)` — not `COUNT(*)`.

Q. Can you use a column alias in `GROUP BY` or `HAVING`? Not in Oracle (through 21c); repeat the expression. You can in `ORDER BY`.

Q. What do `ROLLUP` and `CUBE` do? `ROLLUP(a,b)` gives (a,b) , (a) , grand total; `CUBE(a,b)` gives every combination (a,b) , (a) , (b) , grand total. `GROUPING()` marks the subtotal rows.

Q. What is the difference between `AVG(col)` and `SUM(col)/COUNT(*)`? `AVG` ignores NULLs (divides by non-NULL count); `SUM/COUNT(*)` divides by all rows — different when NULLs exist.

Quick recap

- **Aggregates:** `MIN` `MAX` `COUNT` `SUM` `AVG` (`MEDIAN` in Oracle) collapse many rows to one value and **ignore NULLs** (except `COUNT(*)`).
- **GROUP BY** makes one row per group; every `SELECT` column must be grouped or aggregated; it sits between `WHERE` and `ORDER BY`.
- **WHERE** = rows, before grouping, no aggregates; **HAVING** = groups, after aggregation.
- Patterns: duplicates (`GROUP BY ... HAVING COUNT(*) > 1`) · biggest group (`ORDER BY ... FETCH FIRST 1`) · empty groups (`LEFT JOIN + COUNT(child.col)`) · pivots (`SUM(CASE ...)`).
- **ROLLUP** = subtotals + grand total; **CUBE** = every combination; **GROUPING()** flags subtotal rows.